
LABORATÓRIO 19

OBJETOS DINÂMICOS

EXERCÍCIOS DE REVISÃO

VOCÊ DEVE ACOMPANHAR PARA REVISAR OS CONCEITOS IMPORTANTES

1. Implemente a classe Peixe de forma que ela utilize a classe String desenvolvida nas últimas aulas. Faça com que os métodos de String e de Peixe exibam mensagens toda vez que for realizada a alocação ou liberação de memória.

```
class Peixe
{
private:
    String nome; // tipo do peixe
    float peso; // peso em quilos

public:
    Peixe(const char peixe[], float quilos);
    Peixe(const Peixe& p);
    Peixe & operator=(const Peixe & p);
    ~Peixe();

    // entrada e saída
    friend ostream & operator<<(ostream & os, const Peixe & p);
    friend istream & operator>>(istream & is, Peixe & p);

    // comparação
    friend bool operator<(const Peixe & p1, const Peixe & p2);
    friend bool operator>(const Peixe & p1, const Peixe & p2);
    friend bool operator==(const Peixe & p1, const Peixe & p2);
};
```

Crie um programa que realize alocação de memória (usando new) e observe as mensagens emitidas quando é feita a criação de:

- a) Um único peixe
- b) Um vetor de Peixes

O que acontece se esquecermos de dar o delete?

EXERCÍCIOS DE FIXAÇÃO

VOCÊ DEVE FAZER OS EXERCÍCIOS PARA FIXAR O CONTEÚDO

1. Crie uma classe `Pescado` para gerenciar vários peixes. A classe deve guardar os peixes em memória alocada dinamicamente. Como a classe vai armazenar uma grande quantidade de informações, desabilite o construtor de cópia e operador de atribuição de forma a não permitir que essas operações sejam realizadas em cima de objetos da classe `Pescado`.

```
#include "String.h"
#include "Peixe.h"

class Pescado
{
private:
    Peixe * carga;
    int qtd;

public:
    Pescado(int quantidade);
    Pescado(const Pescado& s) = delete;
    Pescado & operator=(const Pescado& p) = delete;
    ~Pescado();
};
```

Construa um programa para testar a classe `Pescado`, realizando a criação:

- a) De um objeto `Pescado`
- b) De um objeto `Pescado` alocado dinamicamente
- c) De um vetor de `Pescados`
- d) De um vetor de `Pescados` alocado dinamicamente

EXERCÍCIOS DE APRENDIZAGEM

VOCÊ DEVE ESCREVER PROGRAMAS PARA REALMENTE APRENDER

1. Considerando a variante da classe Pilha (Aula 08 – Tipos Abstratos de Dados) mostrada abaixo, implemente seus métodos de forma que ela utilize alocação dinâmica de memória para armazenar seus elementos.

```
using Item = char;

class Stack
{
private:
    enum { MIN_SIZE = 4 };           // tamanho mínimo da pilha
    Item * data;                     // armazenamento dos itens
    int capacity;                     // capacidade atual da pilha
    int size;                         // número de elementos armazenados

    void Expand();                   // expande o armazenamento

public:
    Stack(int n = MIN_SIZE);         // constructor padrão
    Stack(const Stack& st);           // constructor de cópia
    ~Stack();                         // destrutor

    bool isEmpty() const;            // pilha está vazia?
    bool isFull() const;             // pilha está cheia?

    void Push(const Item & item);     // empilha elemento
    const Item Pop();                 // desempilha elemento
    Stack & operator=(const Stack & st); // operador de atribuição
};
```

A pilha deve iniciar com 4 elementos e a sua capacidade deve duplicar cada vez que a operação empilhar se deparar com uma pilha cheia. Adicione testes para verificar o construtor de cópia e operador de atribuição no programa abaixo:

```
#include <iostream>
#include "Stack.h"
using namespace std;

int main()
{
    Stack stack;
    Item item;

    stack.Push('A');
    stack.Push('H');
    stack.Push('L');
    stack.Push('I');
    stack.Push('P');

    while (!stack.isEmpty())
    {
        item = stack.Pop();
        cout << item;
    }
}
```